

Grammar

Set of rules which are used to generate the string is called as grammar.
grammar has a four tuple.

$$G = \{V, T, P, S\}$$

Where, $V \rightarrow$ set of all variables / Non-Terminal / capital letter.

$T \rightarrow$ set of all Terminal / small letter.

$P \rightarrow$ set of all production.

$S \rightarrow$ start symbol.

Foreg :-

1).

$$\begin{array}{ll} S \rightarrow ABC & (r_1) \\ A \rightarrow a & (r_2) \\ B \rightarrow b & (r_3) \\ C \rightarrow c & (r_4) \end{array} \quad \begin{array}{l} V \rightarrow \{S, A, B, C\} \\ T = \{a, b, c\} \\ P = \{r_1, r_2, r_3, r_4\} \\ S = \{S\} \end{array}$$

2).

$$\begin{array}{ll} E \rightarrow E + E & (r_1) \\ E \rightarrow E - E & (r_2) \\ E \rightarrow E = E & (r_3) \\ E = (id) & (r_4) \end{array} \quad \begin{array}{l} V = \{E\} \\ T = \{+, -, =, (,), id\} \\ P = \{r_1, r_2, r_3, r_4\} \\ S = \{E\} \end{array}$$

Note :-

1. grammar is a generating device.
2. If 'G' is the grammar then 'L(G)' is the language generated by grammar.
3. Every Grammar has only one start symbol and derivation of string always start from the start symbol.

Derivation :-

The process of deriving a string is called as derivation.

And the geometrical Representation of derivation is called as

Derivation Tree or ~~path~~ parse Tree.

Ex

$S \rightarrow ABC$

$A \rightarrow a$

$B \rightarrow b$

$C \rightarrow c$

★

$S \rightarrow ABC$

$\rightarrow aBC$

$\rightarrow abc$

$\rightarrow abc$

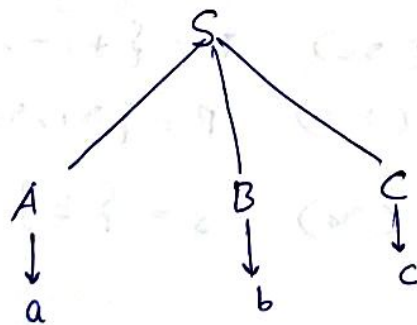
$\rightarrow$ Derivation.

★

$L(G) = \{abc\}$

Language

★



Derivation Tree / Parse Tree

Note :-

1. Every Grammar Generates Only one language
But a language can be generated by
More than one form of grammar So
grammar is not unique.
2. Two grammar ' G_1 ' and ' G_2 ' are equal
if and only if both of them
generate same set of string.

ex

G_1

$$S \rightarrow aA$$

$$A \rightarrow b$$

$$S \rightarrow aA$$

$$\rightarrow ab$$

$$L(G_1) = \{ab\}$$

G_2

$$S \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$S \rightarrow AB$$

$$\rightarrow aB$$

$$\rightarrow ab$$

$$L(G_2) = \{a, b\}$$

Ans

$$G_1 = G_2$$

Find the language Generated by Following

Grammar :-

	<u>grammar</u>	<u>Language generated</u>
1.	$S \rightarrow AB$	$\rightarrow (a+b)B$
	$A \rightarrow a/b$	$= aB + bB$
	$B \rightarrow c/e$	$= a(c+e) + b(c+e)$
		$= ac, a, bc, b$

$$L(G) = \{ac, a, bc, b\}$$

2.	$S \rightarrow aA / bB$	$S \rightarrow (aA + bB)$
	$A \rightarrow b/c$	$= a(b+c) + b(d+e)$
	$B \rightarrow d/e$	$= ab, ac, bd, be$

$$L(G) = \{ab, ac, bd, be\}$$

3.	$S \rightarrow aA Bb$
	$A \rightarrow aB b$
	$B \rightarrow c e$

$$\begin{aligned}
 S &\rightarrow [a(aB + b)] + [(c+e)b] \\
 &= ~~aac + ab~~ \\
 &\quad a[a(c+e) + b] + (c+e)b \\
 &= aac, aa, ab, cb, b
 \end{aligned}$$

$$L(G) = \{aac, aa, ab, cb, b\}$$

4.

$$S \rightarrow Aa \mid bB$$

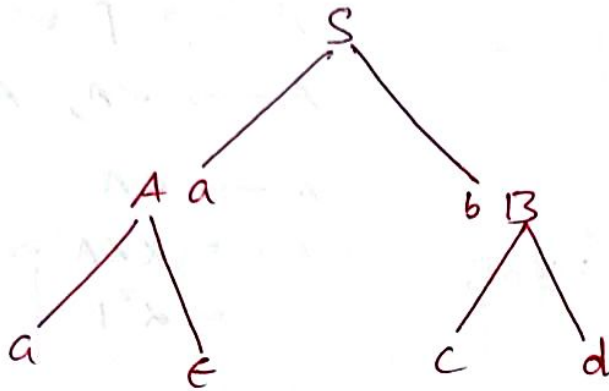
$$A \rightarrow a \mid \epsilon$$

$$B \rightarrow c \mid d$$

$$S = (Aa + bB)$$

$$= (a + \epsilon)a + b(c + d)$$

$$L(G) = \{aa + a + bc + bd\}$$



$$L(G) = \{aa, a, bc, bd\}$$

5.

$$S \rightarrow aA$$

$$A \rightarrow bB \mid a$$

$$B \rightarrow ed \mid \epsilon$$

$$e \rightarrow e$$

$$S = aA$$

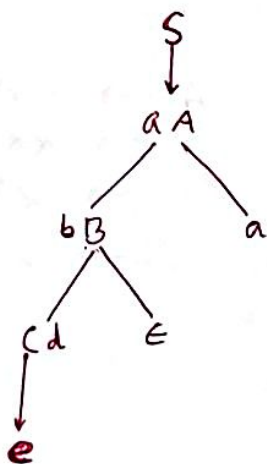
$$= a(bB + a)$$

$$= a(b(ed + \epsilon) + a)$$

$$= a[b(ed + \epsilon) + a]$$

$$= a[bed + b + a]$$

$$L(G) = \{abed + ab + aa\}$$



Top down

left → right.

Que:- Find the language generated by
Following Grammar.

1. $A \rightarrow \alpha A \mid \beta$

$$L(G) = \{\alpha^n \beta\}$$

$$= \{\alpha^n \beta \mid n \geq 0\}$$

$$A \rightarrow \beta \checkmark$$

$$A \rightarrow \alpha A, A \rightarrow \alpha \beta \checkmark$$

$$A \rightarrow \alpha A$$

$$= \alpha \alpha A \text{ or } \alpha \alpha \alpha \beta$$

$$= \alpha^2 \beta$$

$$= \alpha^3 \beta$$

$$= \alpha^4 \beta$$

$$= \alpha^5 \beta$$

$$\boxed{= \alpha^n \beta}$$

2. $A \rightarrow A \alpha \mid \beta$

$$L(G) = \{\beta \alpha^n\}$$

$$A \rightarrow \beta \checkmark$$

$$A \rightarrow A \alpha = \beta \alpha$$

$$= A \alpha \alpha$$

$$= A \alpha^2 = A \alpha^n$$

$$= \beta \alpha^n$$

$$\boxed{= \beta \alpha^n}$$

3. $A \rightarrow A \alpha \mid \alpha$

$$L(G) = \{\alpha \alpha^n\}$$

$$= \{\alpha^+\}$$

$$A \rightarrow A \alpha$$

$$= A \alpha \alpha$$

$$= A \alpha^2$$

$$= \alpha \alpha^n$$

$$\boxed{= \alpha^+}$$

$$(A \rightarrow \alpha)$$

4. $A \rightarrow \alpha A \mid \alpha$

$$L(G) = \{\alpha^n \cdot \alpha\}$$

$$= \{\alpha^+\}$$

$$A \rightarrow \alpha A$$

$$\rightarrow \alpha \alpha A$$

$$\rightarrow \alpha^n A$$

$$\rightarrow \alpha^+$$

$$(A \rightarrow \alpha)$$

$$(\alpha^n \alpha \rightarrow \alpha^+)$$

$$5. \quad A \rightarrow A \alpha \mid \epsilon \quad A = \epsilon \quad \checkmark$$

$$L(G) = \{\alpha^*\}$$

$$\begin{aligned} A &= A\alpha \\ &= A\alpha\alpha \\ &= \epsilon\alpha^2 \\ &= \alpha^2 \\ &= \alpha^* \end{aligned}$$

$$6. \quad S \rightarrow A B \rightarrow a^* b^*$$

$$A \rightarrow a A \mid \epsilon \rightarrow a^*$$

$$B \rightarrow b B \mid \epsilon \rightarrow b^*$$

Soln

$$\begin{aligned} L(G) &= \{a^m b^n \mid m, n \geq 0\} \\ &= \{a^* b^*\} \end{aligned}$$

$$7. \quad S \rightarrow a A \rightarrow a b^*$$

$$A \rightarrow b A \mid \epsilon \rightarrow b^*$$

$$\begin{aligned} L(G) &= \{a b^*\} \\ &= \{a b^n \mid n \geq 0\} \end{aligned}$$

$$8. \quad S \rightarrow a S b \mid \epsilon$$

$$= a(a S b)b$$

$$= a(a(a S b)b)b$$

$$= a^* b^*$$

$$\begin{aligned} L(G) &= \{a^n b^n\} \\ &= \{a^n b^n \mid n \geq 0\} \end{aligned}$$

$$9. \quad S \rightarrow a S \mid b S \mid \epsilon$$

$$S = (a+b)^* S \mid \epsilon$$

$$L(G) = (a+b)^*$$

$$10. S \rightarrow aSa \mid bSb \mid \epsilon$$

$$\rightarrow aSa + bSb$$

$$\rightarrow aasaa + bbsbb$$

$$\rightarrow aaasaaa + bbbSbbbb$$

$$\rightarrow a^* + b^*$$

Language to Grammar :-

Que :- construct the grammar to the following language.

$$1. L = \{a, ab\}$$

$$S \rightarrow ax$$

$$x \rightarrow b \mid \epsilon$$

$$2. L = \{ \underbrace{aab}_x, \underbrace{aba}_x, \underbrace{bab}_x \}$$

$$S \rightarrow x \mid xa \mid bx$$

$$x \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$3. L = \left\{ \frac{aab}{x}, \frac{aba}{x}, \frac{abb}{x} \right\}$$

$$S \rightarrow ax \mid xa \mid xb$$

$$x \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$4. L = \left\{ \frac{aba}{x}, \frac{bab}{x}, \frac{bba}{y}, \frac{abab}{x \ x}, \frac{ba}{y} \right\}$$

$$S \rightarrow xa \mid bx \mid by \mid xx \mid y$$

$$x \rightarrow AB$$

$$y \rightarrow BA$$

$$A \rightarrow a$$

$$B \rightarrow b$$

Que :- Construct the grammar that generate all the string of a's and b's.

1. including ϵ

2. Excluding ϵ

Solution :-

$$1. L = \Sigma^*$$

$$S \rightarrow aS \mid bS \mid \epsilon$$

$$2. L = \Sigma^+$$

$$S \rightarrow aS \mid bS \mid a \mid b$$

Ques:- Construct the grammar that generates All string of a's and b's where every string.

1. start with 'a'
2. start with 'a b'
3. ends with 'b'
4. ends with 'ba'
5. contain the substring 'ab'.

Solution

1. $w = a x$
 $S \rightarrow a x$
 $x \rightarrow a x | b x | \epsilon$

$w = a x$
 $x \rightarrow a x | b x | \epsilon$

2. $w = a b x$
 $S \rightarrow a b x$
 $x \rightarrow a x | b x | \epsilon$

$w = a b x$
 $S \rightarrow a b x$
 $S \rightarrow a y$
 $y \rightarrow b S$
 $S \rightarrow a x | b x | \epsilon$

3. $w = x b$
 $S \rightarrow x b$
 $x \rightarrow a x | b x | \epsilon$

$w = x b$
 $S \rightarrow x b$
 $x \rightarrow a x | b x | \epsilon$

4. $w = S b a$
 $S \rightarrow S b a$
 $S \rightarrow a S | b S | \epsilon$

5. $\frac{X a}{A} \frac{b x}{B}$
 $S \rightarrow A B$
 $A \rightarrow x a$
 $B \rightarrow b x$
 $X \rightarrow a x | b x | \epsilon$

B.N.F (Bacus Naur Form): —

1. Combining the R.H.S of more than one production whenever they are having same non-terminal at L.H.S., is called as B.N.F.

Ex

$$\left. \begin{array}{l} A \rightarrow \alpha_1 \\ A \rightarrow \alpha_2 \\ A \rightarrow \alpha_3 \\ \vdots \\ A \rightarrow \alpha_n \end{array} \right\} \text{Then } A \rightarrow \alpha_1 | \alpha_2 | \alpha_3 | \dots | \alpha_n |$$

Recursive production: —

The production which have same variable / non-terminal at both side L.H.S. and R.H.S. is called Recursive production.

Recursive grammar: —

The grammar with at least one recursive production is called as Recursive grammar.

Ex

$$S \rightarrow a S b$$

$$A \rightarrow a$$

$$B \rightarrow b$$

Non-Recursive grammar :-

The Grammar which is Free From Recursive production is called as Non-Recursive Grammar.

Note :-

1. The Grammar 'G' is Recursive if & only if, it generate infinite No. of string.
2. The grammar 'G' is Non-Recursive if & only if it generate finite No. of string.

Question :- Construct the grammar that generate all the string of a's and b's where every string -

1. Start and end with a
2. start and end with same symbol.
3. start and end with diff. symbol.

Solution :-

1. $w = a x a, a$
 $S \rightarrow a x a \mid a$
 $x \rightarrow a x \mid b x \mid \epsilon$

11. $w = a x a, a$
 $b x b, b$

$$S \rightarrow a x a \mid a \mid b x b \mid b$$

$$x \rightarrow a x \mid b x \mid \epsilon$$

111. $w = a x b$
 $b x a$

$$S \rightarrow a x b \mid b x a$$

$$x \rightarrow a x \mid b x \mid \epsilon$$

Question:- Construct the grammar that generate all the string of a's and b's

where

- i. 3rd symbol from left end is 'a'
- ii. 4th symbol from left end is 'b'
- iii. 3rd symbol " right " " 'a'
- iv. 4th " " " 'b'

Solution:-

1. $w = \underset{\substack{\downarrow \\ (a+b)}}{x} \underset{\substack{\downarrow \\ (a+b)}}{x} a \underset{\substack{\downarrow \\ (a+b)^+}}{x}$
 $\frac{(a+b)^2}{A} a \frac{(a+b)^+}{B}$

$$S \rightarrow A a B$$

$$A \rightarrow x x$$

$$x \rightarrow a \mid b$$

$$B \rightarrow a B \mid b B \mid \epsilon$$

II. $w =$

$$\begin{array}{ccccccc} x & x & x & b & x & & \\ \downarrow & \downarrow & \downarrow & & \downarrow & & \\ (a+b) & (a+b) & (a+b) & & (a+b) & & \end{array}$$

$$\frac{(a+b)^3}{A} \quad b \quad \frac{(a+b)^*}{B}$$

$$\begin{aligned} S &\rightarrow A b B \\ A &\rightarrow XXX \\ X &\rightarrow a|b \\ B &\rightarrow aB | bB | \epsilon \end{aligned}$$

III.

$$\begin{array}{ccccccc} x & a & x & x & & & \\ \downarrow & & \downarrow & \downarrow & & & \\ (a+b)^* & & (a+b) & (a+b) & & & \end{array}$$

$$\frac{(a+b)^*}{A} \quad a \quad \frac{(a+b)^2}{B}$$

$$\begin{aligned} S &\rightarrow A a B \\ B &\rightarrow XX \\ X &\rightarrow a|b \\ A &\rightarrow aA | bA | \epsilon \end{aligned}$$

IV.

$$\begin{array}{ccccccc} x & b & x & x & x & & \\ \downarrow & & \downarrow & \downarrow & \downarrow & & \\ (a+b)^* & & (a+b) & (a+b) & (a+b) & & \end{array}$$

$$\frac{(a+b)^*}{A} \quad b \quad \frac{(a+b)^3}{B}$$

$$\begin{aligned} S &\rightarrow A b B \\ B &\rightarrow XXX \\ X &\rightarrow a|b \\ A &\rightarrow aA | bA | \epsilon \end{aligned}$$

Que:- Construct the grammar For the following language.

a) $L = \{a^m b^n \mid m, n \geq 0\}$

b) $L = \{a^m b^n \mid m, n \geq 1\}$

c) $L = \{a^m b^n \mid m \geq 0, n \geq 1\}$

d) $L = \{a^m b^n \mid m+n = \text{even}\}$

e) $L = \{a^m b^n \mid m+n = \text{odd}\}$

Solution :-

1. $\frac{a^+}{A} \frac{b^+}{B}$

$$S \rightarrow AB$$

$$A \rightarrow aA \mid \epsilon$$

$$B \rightarrow bB \mid \epsilon$$

2. $\frac{a^+ b^+}{A B}$

$$S \rightarrow AB$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid b$$

3. $\frac{a^+}{A} \frac{b^+}{B}$

$$S \rightarrow AB$$

$$A \rightarrow aA \mid \epsilon$$

$$B \rightarrow bB \mid b$$

4. $\frac{a^+}{A} \frac{b^+}{B}$

$$S \rightarrow AB$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow bB \mid \epsilon$$

5. For even - $\frac{(aa)^+}{A} \frac{(bb)^+}{B} + \frac{(aa)^+}{A} a \frac{(bb)^+}{B} b$

$$S \rightarrow AB \mid AaBb$$

$$A \rightarrow aaA \mid \epsilon$$

$$B \rightarrow bbB \mid \epsilon$$

6. For odd -

$$\frac{(aa)^*}{A} a \frac{(bb)^*}{B} + \frac{(aa)^*}{A} b \frac{(bb)^*}{B}$$

$$S \longrightarrow AaB \mid AbB$$

$$A \longrightarrow aaA \mid \epsilon$$

$$B \longrightarrow bbB \mid \epsilon$$

Que :- Construct the grammar For the Following language.

1. $L = \{a^m b^n c^p \mid m, n, p \geq 0\}$

2. $L = \{a^m b^n c^p \mid m = n\}$

3. $L = \{a^m b^n c^p \mid n = p\}$

4. $L = \{a^m b^n c^p \mid m = n \text{ and } n = p\}$

5. $L = \{a^m b^n c^p \mid m = p\}$

6. $L = \{a^m b^n c^p \mid n = m + p\}$

Solution :-

1. $\frac{a^*}{A} \frac{b^*}{B} \frac{c^*}{C}$

$$S \longrightarrow ABC$$

$$A \longrightarrow aA \mid \epsilon$$

$$B \longrightarrow bB \mid \epsilon$$

$$C \longrightarrow cC \mid \epsilon$$

2. $a^m b^n c^p$, Where $m=n$.

$$\frac{a^m}{A} \frac{b^n c^p}{B}$$

$$S \rightarrow AB$$

$$A \rightarrow aAb \mid \epsilon$$

$$B \rightarrow cB \mid \epsilon$$

3. $a^m b^n c^p$ where $n=p$

$$\frac{a^m}{A} \frac{b^n c^p}{B}$$

$$S \rightarrow AB$$

$$A \rightarrow aA \mid \epsilon$$

$$B \rightarrow bB \mid \epsilon$$

↓
small c

4. $L = \{ a^m b^n c^p \}$

$m=n$

$n=p$

$$\frac{a^m}{A} \frac{b^n c^p}{B}$$

$$\frac{a^m}{C} \frac{b^n c^n}{D}$$

$$S \rightarrow AB \mid CD$$

$$A \rightarrow aAb \mid \epsilon$$

$$B \rightarrow cB \mid \epsilon$$

$$C \rightarrow aC \mid \epsilon$$

$$D \rightarrow bDc \mid \epsilon$$

$$\checkmark 5. L = \{ \underbrace{a^m}_{\overline{A}} \underbrace{b^n}_{\overline{B}} \underbrace{c^p}_{\overline{A}} \mid m=p \}$$

$$S \rightarrow ABA$$

$$A \rightarrow aA \mid cA \mid \epsilon$$

$$B \rightarrow bB \mid \epsilon$$

or

$$S \rightarrow aSc \mid A$$

$$A \rightarrow bA \mid \epsilon$$

$$\checkmark 6. L = \{ a^m b^n c^p \mid n = m+p \}$$

$$= \{ a^m b^{m+p} c^p \}$$

$$\underbrace{a^m b^m}_{\overline{A}} \underbrace{b^p c^p}_{\overline{B}}$$

$$S \rightarrow AB$$

$$A \rightarrow aAb \mid \epsilon$$

$$B \rightarrow bBc \mid \epsilon$$

Chomsky Hierarchy :-

Based on the form of production the grammar is classified into Four types.

1. Type zero OR. REG.
2. Type one or CSG.
3. Type two or CFG.
4. Type three or RLG.

1. Type Zero or REG :-

The grammar 'G' said to be type zero (0) or REG if every production is in the form of $\alpha \rightarrow \beta$ where $(\alpha + \beta) \in (V + T)^*$

✓ α means. $(\alpha, \beta) \in (V + T)^*$

variables. $\leftarrow$ $\downarrow$ Terminal

✓ Imp Where at least one variable must be there in L.H.S. of every production.

EX

$$\begin{aligned} S &\rightarrow aB \\ abA &\rightarrow aAb \\ bAB &\rightarrow A ba \\ \underbrace{\quad}_{(V+T)^*} &\quad \underbrace{\quad}_{(V+T)^*} \end{aligned}$$

$$\begin{aligned} \alpha &\rightarrow \beta \\ \text{where} \\ \alpha &= (V+T)^+ \vee (V+T)^* \\ \beta &= (V+T)^* \end{aligned}$$

2. Type one (1) or CSG :-

The Grammar 'G' is said to be type one (1) or CSG, if every production is in the form of

$$\checkmark \alpha \longrightarrow \beta, \text{ where } |\alpha| \leq |\beta|$$

$$\checkmark \text{ and, } \beta \not\rightarrow \epsilon$$

$$\checkmark \text{ and, } (\alpha, \beta) \in (V + T)^*$$

$\downarrow$ variable $\downarrow$ Terminal

ex

$$S \longrightarrow aBa$$

$$ab \longrightarrow bab$$

$$bab \longrightarrow abbaB$$

$\checkmark$ • CSG does not contain Any production To generate empty string ' ϵ '

3. Type two (2) or CFG :-

The grammar 'G' is said to be type '2' or CFG if every production is in the form of $A \rightarrow \alpha$, where

$$A \in V \leftarrow \begin{matrix} \text{single} \\ \text{variable} \end{matrix}$$

$$\alpha \in (V + T)^*$$

ex $S \rightarrow aSb \mid \epsilon$

$$B \rightarrow aBA \mid b$$

$$A \rightarrow abB \mid b$$

$$A \in |V| = 1$$

imp Note :- Every programming Language is defined by CFG.

4. Type three (3) or RG :-

The grammar 'G' is said to be type '3' or Regular if every production is in the form $A \rightarrow Bx \mid y$

or $A \rightarrow xB \mid y$
Where, $A, B \in V \rightarrow$ ^{single} variable.
 $x, y \in T^* \rightarrow$ Terminal.

Ex

$S \rightarrow Aaa \mid bb$

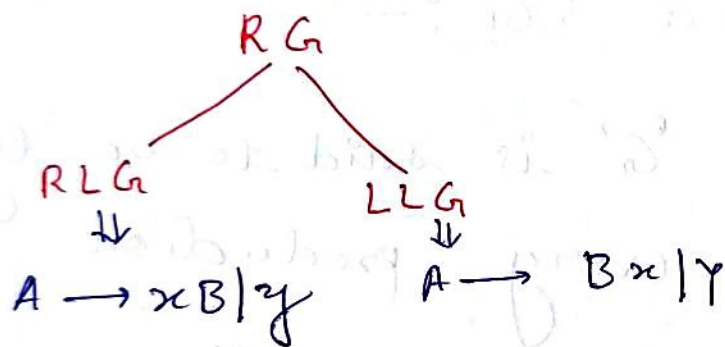
$A \rightarrow B ab \mid ba$
or

$S \rightarrow aaA \mid bb$

$A \rightarrow abB \mid ba$

Types of Regular Grammar :-

1. $RLG \longrightarrow$ Right linear Grammar.
2. $LLG \longrightarrow$ Left linear Grammar.



Note :-

1. Every Regular Language can be generated by either by 'RLG' or 'LLG'.

Regular Grammar

Right linear Grammar

$$A \rightarrow xB$$

$$A \rightarrow x$$

where $A, B \in V$
 $x \in T$

Left linear Grammar

$$A \rightarrow Bx$$

$$A \rightarrow x$$

where $A, B \in V$
 $x \in T$

example 1. $S \rightarrow abS \mid b$

eg $S \rightarrow Sb \mid b$

Derivation of Grammar :

Prob 1: $G = (\{S, A\}, \{a, b\}, S, \{S \rightarrow aAb, aA \rightarrow aaAb, A \rightarrow \epsilon\})$

Sol

$$\begin{aligned} S &\rightarrow aAb && [\text{by } S \rightarrow aAb] \\ &\rightarrow aaAbb && [\text{by } aA \rightarrow aaAb] \\ &\rightarrow aaaSbbb && [\text{by } aA \rightarrow aaAb] \\ &\rightarrow aaaSbbb && [\text{by } A \rightarrow \epsilon] \\ L(G) &= \{aaaSbbb\} \end{aligned}$$

Prob 2: $G = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\})$

Sol

$$\begin{aligned} S &\rightarrow AB && [\text{by } S \rightarrow AB] \\ &\rightarrow aB && [\text{by } A \rightarrow a] \\ &\rightarrow ab && [\text{by } B \rightarrow b] \\ L(G) &= \{ab\} \end{aligned}$$

Prob 3: $G = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow aA \mid a, B \rightarrow bB \mid b\})$

Sol

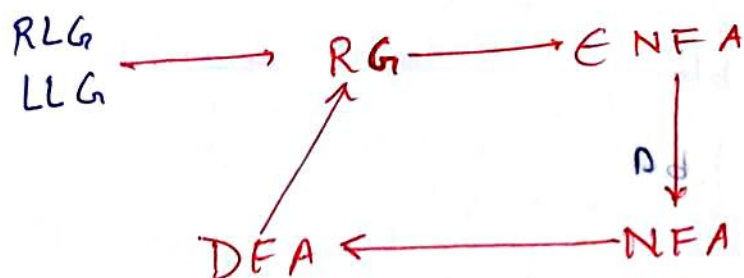
$$\begin{aligned} S &\rightarrow AB \\ &\rightarrow aB \\ &\rightarrow ab \end{aligned}$$

$$\begin{aligned} S &\rightarrow AB \\ &\rightarrow aAB \\ &\rightarrow aab \end{aligned}$$

$$\begin{aligned} S &\rightarrow AB \\ &\rightarrow aAB \\ &\rightarrow aab \end{aligned}$$

$$\begin{aligned} S &\rightarrow AB \\ &\rightarrow aBB \\ &\rightarrow abb \end{aligned}$$

* Regular Grammar to ϵ NFA :-

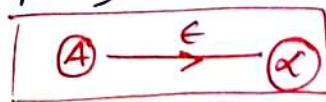


* 1. Conversion of RLG to ϵ NFA :-

* Procedure :-

1. start symbol is the initial state
2. If $A \rightarrow \alpha$ then $\delta(A, \epsilon) = \alpha$.

[152]



$$\alpha \xrightarrow{a} \alpha$$

$$\delta(a, a) = \odot$$

4.

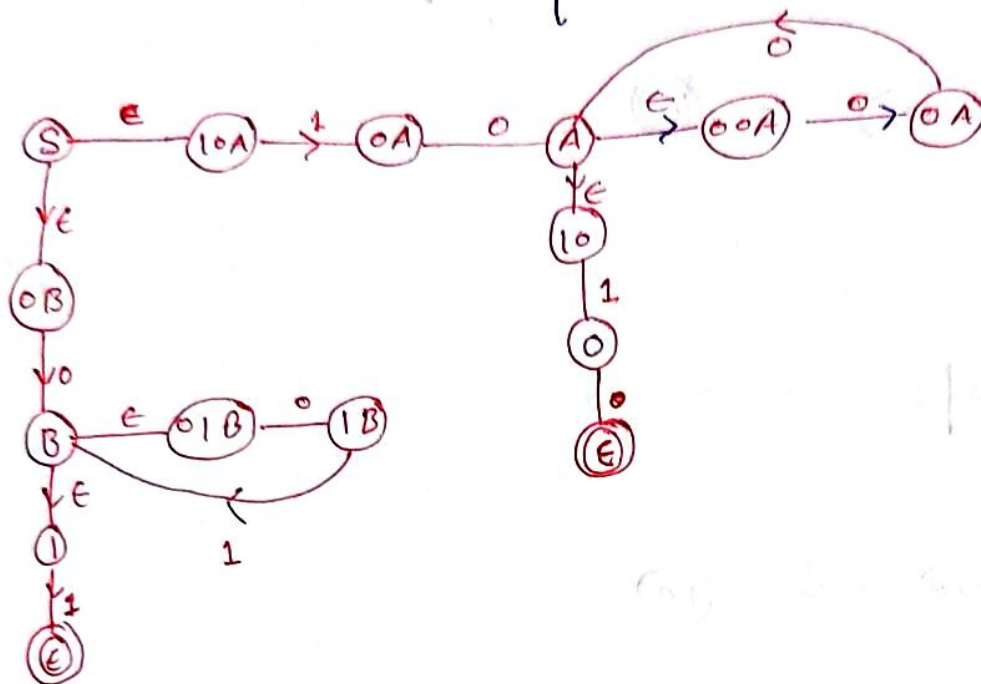
```
graph LR; A((A)) -- epsilon --> 01A((01A)); A -- epsilon --> 1((1)); 01A -- 0 --> 1A((1A)); 1A -- 1 --> A; 1 -- 1 --> E(((E)))
```

[illegible]

A hand-drawn Non-deterministic Finite Automaton (NFA) diagram for the regular expression 0^*10^* . The diagram consists of the following states and transitions:

- States:**
 - Start state: A (circle with an incoming arrow from the left).
 - Intermediate state 1: $00|A$ (circle).
 - Intermediate state 2: $0|A$ (circle).
 - Intermediate state 3: $1A$ (circle).
 - Intermediate state 4: 10 (circle).
 - Final state: ϵ (double circle, reached from 10 via a red arrow).
- Transitions:**
 - $A \xrightarrow{\epsilon} 00|A$
 - $00|A \xrightarrow{0} 0|A$
 - $0|A \xrightarrow{0} 1A$
 - $1A \xrightarrow{1} A$ (top curved transition)
 - $A \xrightarrow{\epsilon} 10$
 - $10 \xrightarrow{\epsilon} \epsilon$ (red arrow)

4. $S \rightarrow 10A \mid 0B$
 $A \rightarrow 00A \mid 10$
 $B \rightarrow 01B \mid 1$



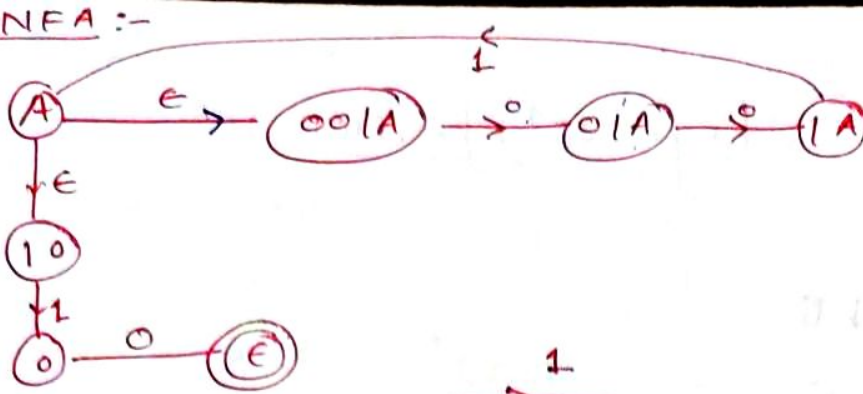
2. Conversion of L.L.G. To ϵ NFA :-

1. Reverse R.H.S. of every production.
2. Obtain ϵ NFA
3. Interchange initial and Final state
4. Change the direction of edges.

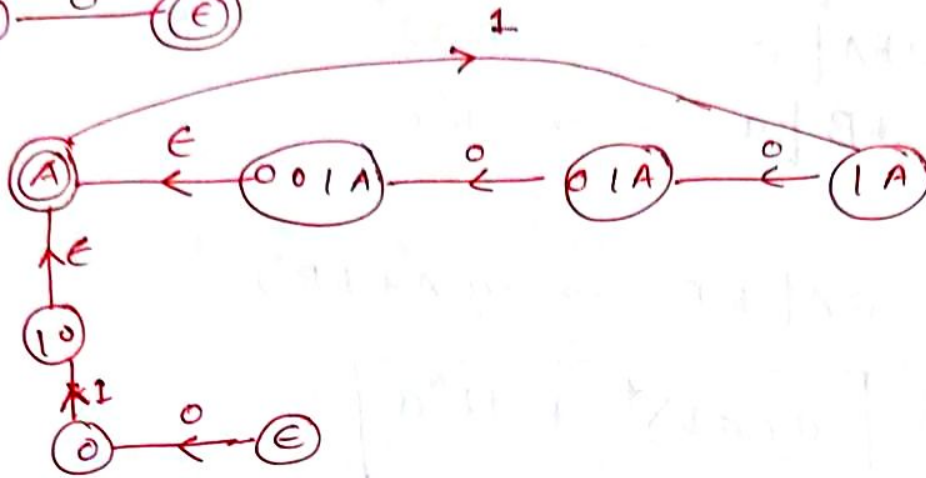
Ex:- $A \rightarrow A100 \mid 01$

1. Reversing $A \rightarrow 001A \mid 10$

2. ENFA :-

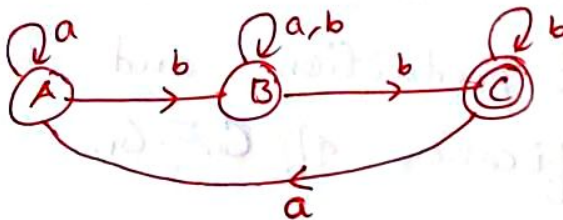


3.



Equivalence b/w F.A. and Regular Grammar :-

Ex



$A \rightarrow aA \mid bB$

$B \rightarrow aB \mid bB \mid bC$

$C \rightarrow aA \mid bC \mid \epsilon$

Regular Grammar To Regular Expression :-

Ex

1. $S \longrightarrow aA \mid bB$

$$A \longrightarrow abA \mid \epsilon \longrightarrow (ab)^*$$

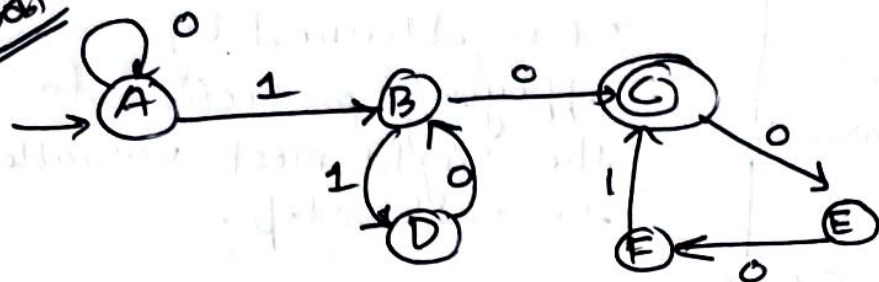
$$B \longrightarrow bB \mid a \longrightarrow b^*a$$

$$S \longrightarrow aA \mid bB \longrightarrow (aA + bB)$$

$$R.E. \longrightarrow \boxed{a(ab)^* + bb^*a}$$

Conversion of Finite Automata to Grammar

Prob 1



Consider all outgoing transitions on each given state.

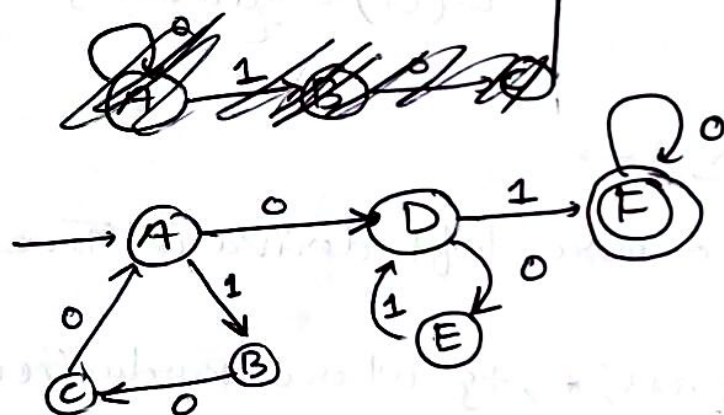
RLG

$A \rightarrow 0A \mid 1B$
 $B \rightarrow 0C \mid 1D$
 $C \rightarrow 0E \mid \epsilon$
 $D \rightarrow 0B$
 $E \rightarrow 0F$
 $F \rightarrow 1C$

LLG

$A \rightarrow A0 \mid B1$
 $B \rightarrow C0 \mid D1$
 $C \rightarrow E0 \mid \epsilon$
 $D \rightarrow B0$
 $E \rightarrow F0$
 $F \rightarrow C1$

Prob 2



RLG

$A \rightarrow 0D \mid 1B$
 $B \rightarrow 0C$
 $C \rightarrow 0A$
 $D \rightarrow 0E \mid 1F$
 $E \rightarrow 1D$
 $F \rightarrow 0F \mid \epsilon$

LLG

$A \rightarrow D0 \mid B1$
 $B \rightarrow C0$
 $C \rightarrow A0$
 $D \rightarrow E0 \mid F1$
 $E \rightarrow D1$
 $F \rightarrow F0 \mid \epsilon$

CFG :

$A \rightarrow a$
where $a = \{V U \in \Sigma^+\}$ and $A = |V| = 1$
or
 $a = \{V U \in \Sigma^+\}$

Prob $G = \{(S, A), (a, b), S \rightarrow aAb, A \rightarrow aAb | \epsilon\}$

Sol

$S \rightarrow aAb$
 $\rightarrow aaAbb$ (by $A \rightarrow aAb$)
 $\rightarrow aaaaAbbb$ (" "
 $\rightarrow aaaa bbb$ (by $A \rightarrow \epsilon$)
 $\rightarrow a^3 b^3 \Rightarrow a^n b^n$

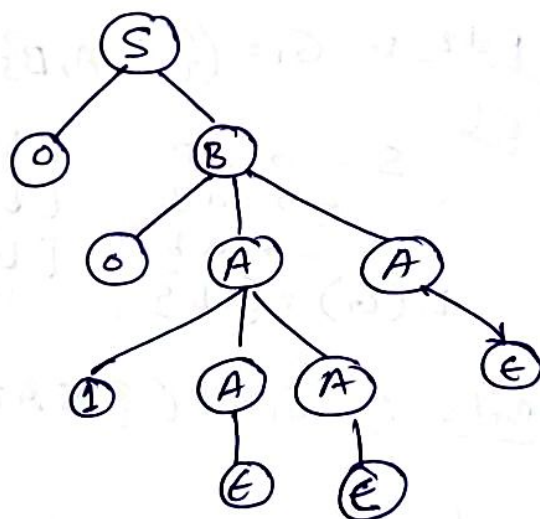
Derivation Tree : —

Derivation Tree or Parse Tree is ordered rooted tree that Graphically represents semantic information of strings derived from CFG.

Prob

$G = \{V, T, P, S\}$

$S \rightarrow 0B$
 $A \rightarrow A1AA | \epsilon$
 $B \rightarrow 0AA$



Root Vertex $\rightarrow$ Start Symbol

Vertex $\rightarrow$ Non-Terminal/Variable

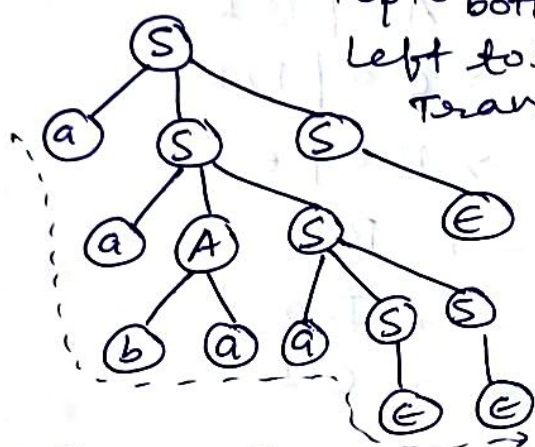
Leaves $\rightarrow$ Terminals

Left Derivation

Tree of CFG
It is obtained by Applying production to the left most variable in each step.

Prob $S \rightarrow aAS \mid aSS \mid \epsilon$
 $A \rightarrow \bullet SBA \mid ba$

Top to bottom
Left to right
Traverse

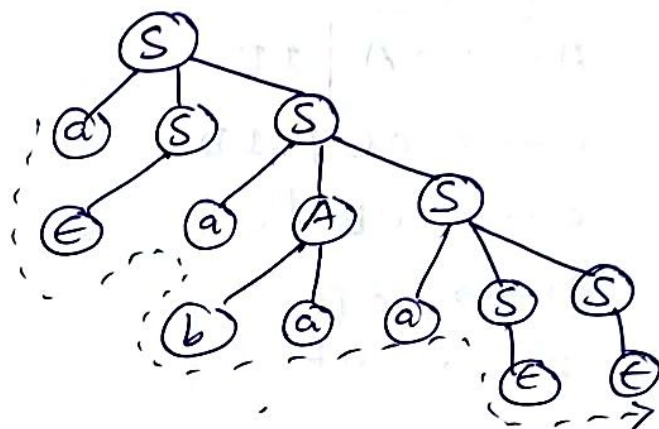


$L(G) = \{aaba\}$

Right Derivation

Tree of CFG

It is obtained by applying production to the right most variable in each step.



$L(G) = \{aaba\}$

Ambiguous Grammar $\rightarrow$

If there exist two or more left derivation Tree for a string of a grammar.

Prob $G = (\{S\}, \{a+b\}^*, +)$ where productions are

$S \rightarrow S+S \mid S * S \mid a \mid b$

The string $a+a*b$ can be generated as:

$$\begin{aligned} S &\rightarrow S+S \\ &\rightarrow a+S \\ &\rightarrow a+S*S \\ &\rightarrow a+a*S \\ &\rightarrow a+a*b \end{aligned}$$

$$\begin{aligned} S &\rightarrow S * S \\ &\rightarrow S + S * S \\ &\rightarrow a + S * S \\ &\rightarrow a + a * S \\ &\rightarrow a + a * b \end{aligned}$$

this grammar is Ambiguous.

* Simplification of CFG :-

The process of detection and elimination of useless symbol, unit production and ' ϵ ' production is called as simplification of C.F.G.

* useless symbol :-

The Non-Terminal which does not involve in the derivation of string is called as useless symbol.

* Note :-
*1. Every ^{from start.} unreachable symbol is useless symbol.

*2. The symbol which is reachable from start symbol but does not derive any Terminal is called as useless symbol.

Ex

1. $S \rightarrow Aa$
 $A \rightarrow b$
 $B \rightarrow a$ ~~X~~ useless symbol
 $\Downarrow$

$S \rightarrow Aa$
 $A \rightarrow b$ } simplified C.F.G.

2. $S \rightarrow aA \mid aA\bar{B} \rightarrow$ useless symbol.
 $A \rightarrow b$
 $\Downarrow$
 $S \rightarrow aA$
 $A \rightarrow b$ } simplified C.F.G.

3. $S \rightarrow aAB \mid aB\bar{C} \rightarrow$ useless symbol.
 $A \rightarrow \bar{a}\bar{C} \mid bB \rightarrow$ useless symbol.
 $B \rightarrow aB \mid bA \mid a$
 $\Downarrow$

$S \rightarrow aAB \mid$
 $A \rightarrow bB$
 $B \rightarrow aB \mid bA \mid a$

* Unit Production :-

The Production of the form $A \rightarrow B$
 Where $A, B \in \underline{V}$ is called unit production.
 Variable.

* Note :-

Remove the unit production and Replace the equivalence derivation.

Ex

1.
$$\begin{aligned} S &\rightarrow aA \mid bB \\ A &\rightarrow \textcircled{B} \mid bA \mid a \\ B &\rightarrow bB \mid b \end{aligned}$$

$$\begin{aligned} S &\rightarrow aA \mid bB \\ A &\rightarrow bB \mid b \mid bA \mid a \\ B &\rightarrow bB \mid b \end{aligned}$$

2.
$$\begin{aligned} S &\rightarrow aA \\ A &\rightarrow \underline{B} \mid a \\ B &\rightarrow \underline{C} \mid b \\ C &\rightarrow \underline{D} \mid e \\ D &\rightarrow e \end{aligned}$$

$$\begin{aligned} S &\rightarrow aA \\ A &\rightarrow e \mid b \mid a \\ A &\rightarrow D \mid C \mid b \mid a \end{aligned}$$

$$\begin{aligned} S &\rightarrow aA \\ A &\rightarrow e \mid b \mid a \end{aligned}$$

✱ 'ε' Production :-

Production of the form of $A \rightarrow \epsilon$ is called 'ε' production.

$$\boxed{A \rightarrow \epsilon}$$

is

✱ Note :-

Remove all 'Null' production and replace the "equivalence derivation."

Ex

1. $S \rightarrow aS | \epsilon$
 $\Downarrow$
 $S \rightarrow aS | a$

2. $S \rightarrow AaB \Rightarrow S \rightarrow AaB | Aa$
 $A \rightarrow abA | b \quad A \rightarrow abA | b$
 $B \rightarrow BaA | bB | \epsilon \quad B \rightarrow BaA | bB | aA | b$

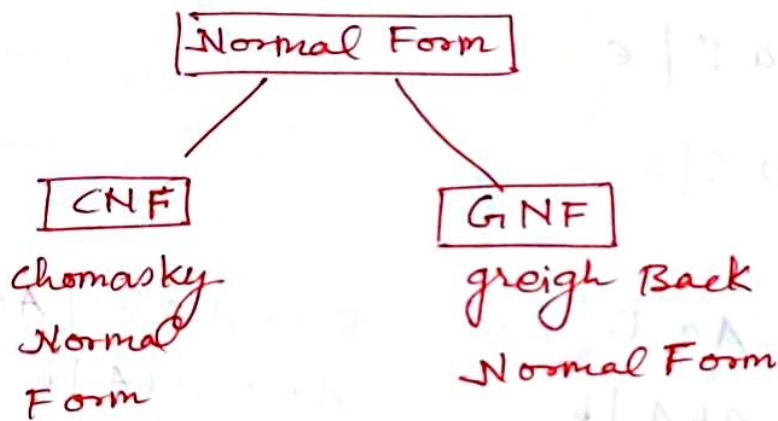
3. $S \rightarrow AaB \Rightarrow S \rightarrow AaB | aB | Aa | a$
 $A \rightarrow aA | \epsilon \Rightarrow A \rightarrow aA | a$
 $B \rightarrow bB | \epsilon \quad B \rightarrow bB | b$

Order of process of simplification :-

1. Elimination of 'ε' production.
2. Elimination of unit production.
3. Elimination of useless symbol.

~~Imp~~ ϵ ^③ < unit production ^② < Useless symbol ^①

Normal Forms :-



Note :-

1. To convert into Normal form the grammar must be Free From 'ε' production.

1. CNF :-

The Grammar, 'G' is said to be 'CNF' if every production in the form of

$$A \rightarrow \underbrace{BC}_{\text{two Non-Terminal}} \mid \underbrace{a}_{\text{one-terminal}} \quad \text{where } A, B, C \in V \rightarrow \text{variables (Non-Terminal)}$$

$$a \in T \rightarrow \text{Terminal.}$$

Ex

1).

$$S \rightarrow aSb \mid \epsilon$$

$$S \rightarrow aSb \mid ab$$

$$S \rightarrow \underbrace{A}_{\text{Y}} SB \mid AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$\Downarrow$

$$S \rightarrow AY \mid AB$$

$$Y \rightarrow SB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

2).

$$S \rightarrow aA \mid bB$$

$$A \rightarrow aAb \mid bB \mid a \Rightarrow$$

$$B \rightarrow bBa \mid Ba \mid b$$

$$S \rightarrow XA \mid YB$$

$$A \rightarrow XAY \mid YB \mid a$$

$$B \rightarrow YBX \mid BX \mid b$$

$$X \rightarrow a$$

$$Y \rightarrow b$$

$\Downarrow$

$$S \rightarrow XA \mid YB$$

$$A \rightarrow XM \mid YB \mid a$$

$$B \rightarrow YN \mid BX \mid b$$

$$M \rightarrow AY$$

$$N \rightarrow BX$$

$$X \rightarrow a$$

$$Y \rightarrow b$$

Ques

$$\begin{aligned} S &\rightarrow A \underline{aA} | B \underline{b} \\ A &\rightarrow a \underline{b} | B a | b \\ B &\rightarrow b B a | B A | a \end{aligned}$$

$$\Rightarrow \begin{aligned} S &\rightarrow \underline{A} X A | B Y \\ A &\rightarrow X Y | B X | b \\ B &\rightarrow Y \underline{B X} | B A | a \end{aligned}$$

$$\Downarrow$$

$$\begin{aligned} S &\rightarrow M A | B Y \\ A &\rightarrow X Y | B X | b \\ B &\rightarrow Y N | B A | a \\ M &\rightarrow A X \\ N &\rightarrow B X \\ X &\rightarrow a \\ Y &\rightarrow b \end{aligned}$$

$\rightarrow \underline{\underline{G.N.F.}}$

The Grammar 'G' is said to be in G.N.F. if Every production is in the form of

$$\boxed{A \rightarrow a B} \quad , \quad \text{Where } a \in T \rightarrow \text{terminal (one Terminal)}$$

$$\underline{A \rightarrow a B} \quad B \in V^+ \rightarrow \text{variables.}$$

EX

$$1. \quad \begin{aligned} S &\rightarrow a S b | \epsilon \\ S &\rightarrow a S b | a b \end{aligned} \Rightarrow \begin{aligned} S &\rightarrow a S B | a B \\ B &\rightarrow b \end{aligned}$$

$$2. \quad \begin{aligned} S &\rightarrow a S b | b S b | \epsilon \\ S &\rightarrow a S b | b S b | a b | b b \end{aligned}$$

$\Downarrow$

$$\begin{aligned} S &\rightarrow a S B | b S B | a B | b B \\ B &\rightarrow b \end{aligned}$$

3>

$$S \rightarrow Ab \mid Bb$$

$$A \rightarrow aA \mid b$$

$$B \rightarrow bBa \mid a$$

$\Rightarrow$

$$S \rightarrow aAb \mid bb \mid bBab \mid ab$$

$$A \rightarrow aA \mid b$$

$$B \rightarrow bBx \mid a$$

$$S \rightarrow aAY \mid bY \mid bBXY \mid aY$$

$$A \rightarrow aA \mid b$$

$$B \rightarrow bBX \mid a$$

$$X \rightarrow a$$

$$Y \rightarrow b$$

4.

$$S \rightarrow aSb \mid \epsilon$$

CNF

GNF

$$S \rightarrow aSb \mid \underline{a}b$$

$$S \rightarrow aS\underline{b} \mid ab$$

$$S \rightarrow A\underline{S}B \mid AB$$

$$S \rightarrow aS\underline{B} \mid aB$$

$$S \rightarrow AY \mid AB$$

$$B \rightarrow b$$

$$Y \rightarrow SB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

aa bb Accepted by these grammars.

BY CNF

$$S \rightarrow AY$$

$$\rightarrow ASB$$

$$\rightarrow aSB$$

$$\rightarrow aAB B$$

$$\rightarrow aaBB$$

$$\rightarrow aabB$$

$$\rightarrow aabb$$

7 Steps.

$$= 2n - 1$$

$n \rightarrow$ length of string

BY GNF

$$S \rightarrow aSB$$

$$\rightarrow aaBB$$

$$\rightarrow aabb$$

$$\rightarrow aabb$$

4 steps

$$= n$$

length of string

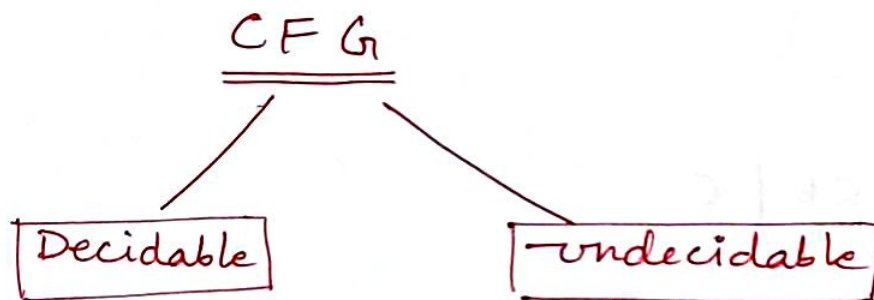
Note :-

No. of production Required to generate string of length 'n' is

In CNF	=	$2n-1$
In GNF	=	n

V. imp

V. imp



- | | |
|----------------|---------------------|
| 1. Emptiness. | 1. Ambiguity |
| 2. Finiteness. | 2. Equality |
| 3. Membership. | 3. Regularity
or |
| | 4. Non-Regularity. |

1. Emptiness :-

1. Remove all the useless symbol.
2. If the Reduced C.F.G. contain atleast one production which is reachable from start symbol. and derives atleast one terminal. Then the grammar generate Non-empty language otherwise grammar generate empty language.

EX :-

1. $S \rightarrow aA \cancel{B} \rightarrow \text{useless.}$
 $A \rightarrow b|a$

$\Downarrow$

$A \rightarrow b|a \rightarrow \text{Empty language.}$

2. $S \rightarrow aA | \cancel{bB} \rightarrow \text{useless.}$

$A \rightarrow aA | \cancel{bB} | \epsilon$

$\Downarrow$

$S \rightarrow aA$

$A \rightarrow aA | \epsilon$

$\Downarrow$

$S \rightarrow aA$

$A \rightarrow a$

} Non-empty language

③ ✓

$S \rightarrow aA | \cancel{bB}$

$A \rightarrow Aa | \cancel{bB} | a$

$B \rightarrow \cancel{aB}$

$\Downarrow$

$S \rightarrow aA$

$A \rightarrow Aa | a$

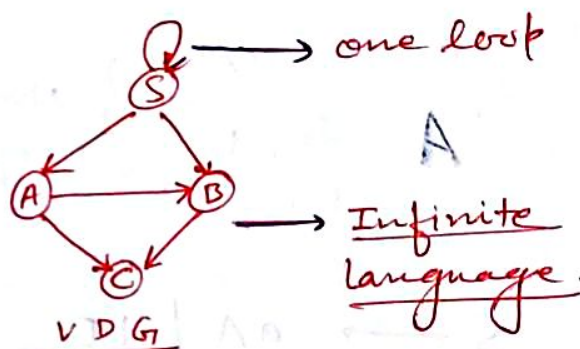
} Non-empty language

2. Finiteness :-

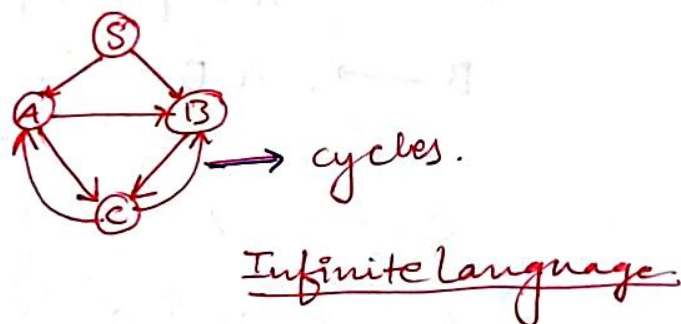
1. Convert the Grammar into CNF.
2. Draw the variable dependency graph (VDG) $\rightarrow$ Variable dependency graph.
3. If V.D.G. contains loop or cycle then the grammar generate Infinite language, If VDG is Free From loop and cycle it generate Finite language.

Ex :-

1. $S \rightarrow AB \mid SS$
 $A \rightarrow BC \mid a$
 $B \rightarrow CC \mid b$



2. $S \rightarrow AB$
 $A \rightarrow BC \mid a$
 $B \rightarrow CC \mid b$
 $C \rightarrow AB$



3. $S \rightarrow AB$
 $A \rightarrow BC \mid a$
 $B \rightarrow CC \mid b$

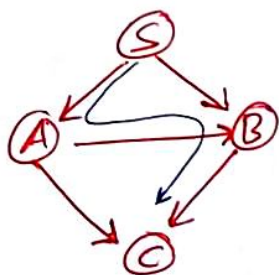


Note :-

1. In V.D.G. We can define rank of each and every non-terminals (variables).
2. Rank of $P(X)$ = length of the "longest path" from 'X' to last node.
3. The grammar generate Infinite language if and only if the rank of atleast one non terminal is infinite.
4. The grammar generate Finite language if the rank of each and every non-terminal is Finite.

Ex

1.



$$P(S) = 3$$

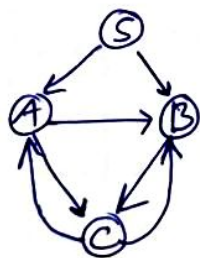
$$P(A) = 2$$

$$P(B) = 1$$

$$P(C) = 0$$

Finite Language

2.



$$P(S) = \infty$$

$$P(A) = \infty$$

$$P(B) = \infty$$

$$P(C) = \infty$$

Infinite Language

3. Membership :-

1. Membership is the property to verify that any string 'w' is generated by Grammar or not.
2. Membership property is implemented by CYK Algorithm.
(Cook's Young Kasami)
3. CYK Algorithm can be implemented if the grammar is in CNF.

Imp 4. CYK Algorithm is the best example of dynamic programming.

Imp 5

No. of Table entries in CYK algo is where $n \rightarrow$ length of strings.

$$\frac{n(n+1)}{2}$$

Ex :-

$S \rightarrow BA$

$A \rightarrow AA \mid SB \mid a$

$B \rightarrow AB \mid AS \mid b$

1. $w = ab$

a	b
A	B
B	

$w \notin L(G)$

2. $w = ba$

b	a
B	A
S	

$w \notin L(G)$

4)

$$S \rightarrow BA$$

$$A \rightarrow AA \mid SB \mid a$$

$$B \rightarrow AB \mid AS \mid b$$

$w = aba$

	a	b	a
A	1	B	2
B	2	S	1
B, S			

$$1 \rightarrow 1 \Rightarrow AS$$

$$2 \rightarrow 2 \Rightarrow BA$$

$$w \in L(G)$$

EX2 $S \rightarrow AB$

$$A \rightarrow BB \mid SA \mid b$$

$$B \rightarrow BA \mid BS \mid a$$

$$w = abaaab$$

	a	b	a	a	b
B	1	A	1	B	1
B	2	S	1	A	1
B, A	3	1	2	A	2
B, A, S	4	1	3	1	4
B, A, S					

$$w \in L(S)$$

abaaab is a member

$w = aab$

	a	a	b
A	1	A	2
A	2	B	1
B, B			

$$1 \rightarrow 1 = AB$$

$$2 \rightarrow 2 = AB$$

$$w \notin L(G)$$

~~4)~~ ~~5)~~

$w = abab$

	a	b	a	b
A	1	B	2	A
B	2	S	1	B
B, S	3	1	2	A
A, A				

$$1 \rightarrow 1 = AS$$

$$2 \rightarrow 2 = BA$$

$$1 \rightarrow 1 = AA$$

$$2 \rightarrow 2 = BB$$

$$3 \rightarrow 3 = BB, SB$$

$$w \notin L(G)$$

Push Down Automata

1. The Mathematical representation of C.F.L. is called as PDA.
2. F.A. With external memory is called as PDA.
3. PDA has seven tuples.

$$PDA = \{ Q, \Sigma, \delta, q_0, F, \Gamma, z_0 \}$$

Where, $Q \longrightarrow$ set of all states

$\Sigma \longrightarrow$ i/p alphabet.

$\delta \longrightarrow$ Transition Function

$q_0 \longrightarrow$ Initial state

$F \longrightarrow$ Final state

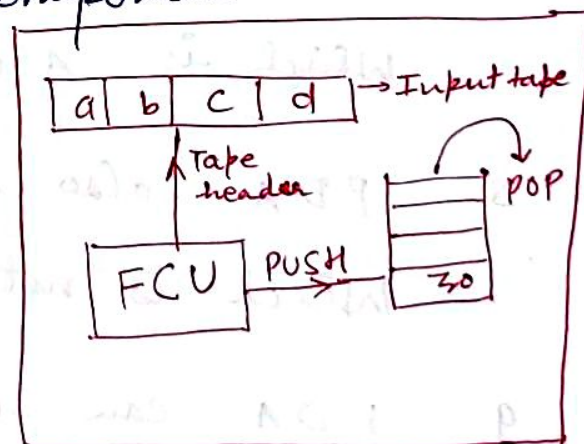
$\Gamma \longrightarrow$ set of all stack symbol.

$z_0 \longrightarrow$ Top most symbol of stack.

Block Diagram of PDA :-

PDA consists of Four component.

1. Input Tape.
2. Tape header
3. Finite Control unit (FCU)
4. Stack



Note :-

1. F.A. + 1 stack = PDA
2. Stack is used by PDA as an external storage location.
3. No change in the basic Functionality of F.A. and P.D.A. except "stack"
4. With addition of stack PDA has got more Accepting power than F.A. But there is no change in the computational power of F.A. and PDA.
5. ' z_0 ' is the stack symbol, and purpose of ' z_0 ' is only to see that the stack is empty or not.
6. PDA is more powerful than F.A.
7. PDA can Accept every language which is Accepted by F.A.
8. PDA also Accept some of the language which is not Accepted by F.A.
9. PDA can be constructed in two models
 1. NPDA
 2. DPDA

~~Imp~~ 10. By default PDA is "NPDA"

11. The language Accepted by PDA is known as CFL

12. ~~Imp~~ Every CFL is Accepted by "NPDA"

13. ~~Imp~~ Some of the CFL is Accepted by "DPDA".

~~Imp~~

NPDA $\neq$ DPDA

~~V.V Imp~~

14.

Every language which is accepted by 'DPDA' is also accepted by NPDA but converse need not be true.

That is Accepting power of DPDA $\neq$ Accepting power of NPDA.

15. The language Accepted by DPDA is called as DCFL.

16. The language Accepted by NPDA is called as "CFL"

17. Every DCFL is a CFL but every "CFL" need not be "DCFL".

18. Let " L_D = set of all languages Accepted by DPDA" and " L_N = set of all language Accepted by NPDA."

$$L_D \subset L_N$$

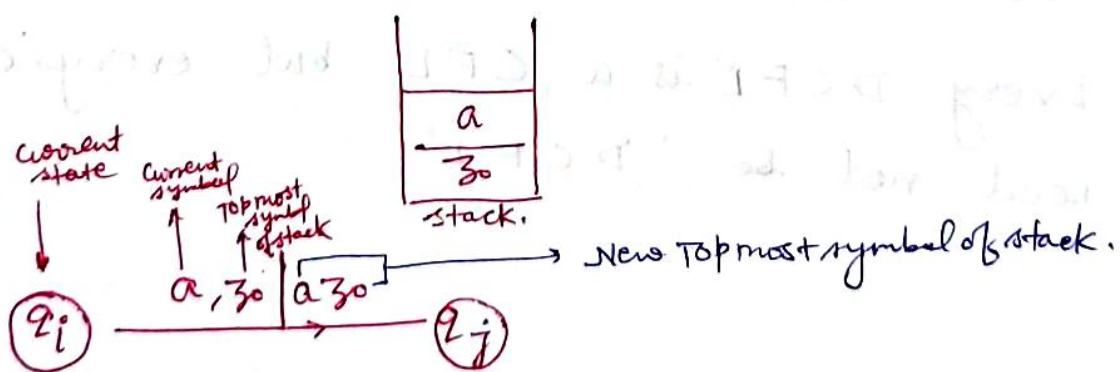
19. NPDA is more powerfull than DPDA.

Instantaneous Description :- (ID) :-

1. ID describe the movement of PDA.
2. The movement PDA depends on three entities
 - Current state.
 - Current processing Symbol.
 - Current Topmost symbol of stack.

$$\delta(q_i, a, z_0) = (q_j, a, z_0)$$

$\downarrow$ Current state $\downarrow$ Current processing symbol $\downarrow$ Current Top most symbol of stack $\downarrow$ New state $\downarrow$ New Top most symbol of stack.



* Acceptance of PDA :-

1. PDA can accept the string in two ways. —

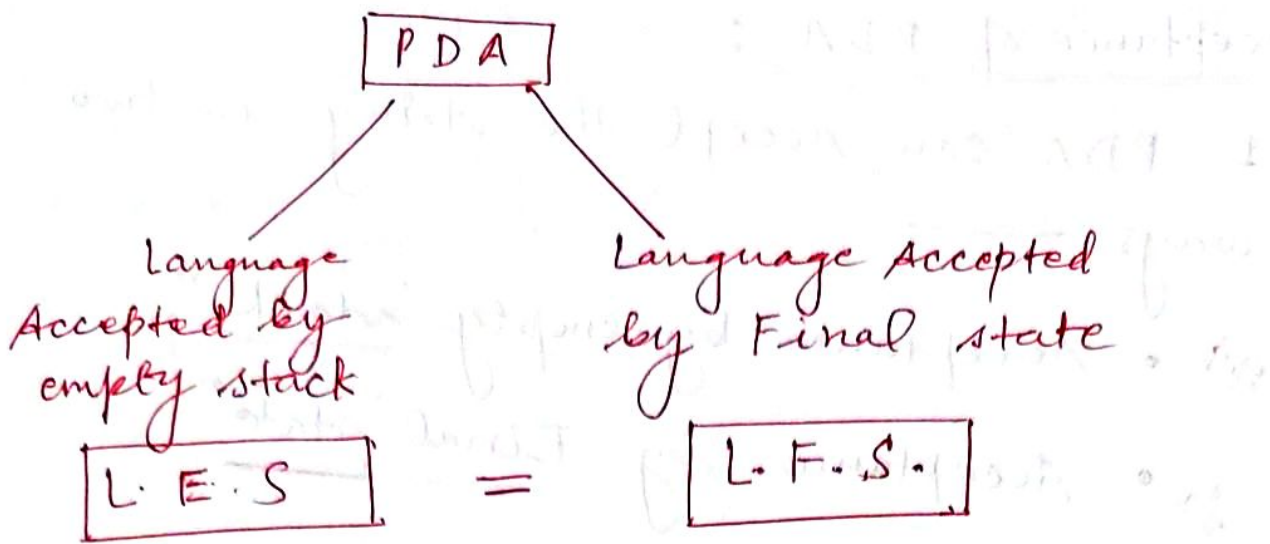
- Imp • Acceptance by empty "stack."
- Imp • Acceptance by Final "state."

1. Acceptance by empty stack :-

After reading the complete input string if stack is empty, then input string is Accepted by PDA. And this Mechanism is called as Acceptance by empty stack.

2. Acceptance by Final state :-

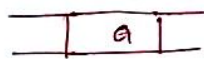
After reading the complete input string if PDA reach to the Final state then the input string is Accepted by PDA and this Mechanism is called as Acceptance by Final state.



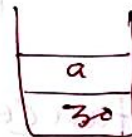
OPERATION IN PDA :-

1. PUSH
2. POP
3. SKIP

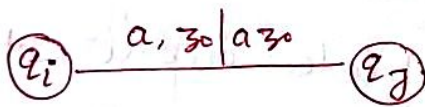
1. PUSH :-



↑ input take

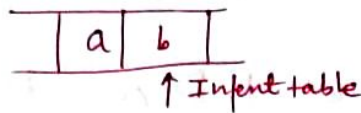


stack.

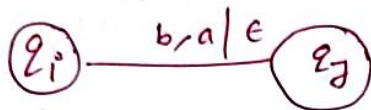
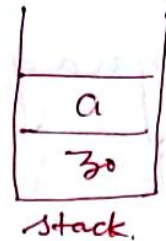


$$\delta \left(\begin{matrix} q_i \\ \downarrow \text{CS} \end{matrix}, \begin{matrix} a \\ \downarrow \text{CPS} \end{matrix}, \begin{matrix} z_0 \\ \downarrow \text{CTS} \end{matrix} \right) = \left(\begin{matrix} q_j \\ \downarrow \text{NS} \end{matrix}, \begin{matrix} a z_0 \\ \downarrow \text{NTS} \end{matrix} \right)$$

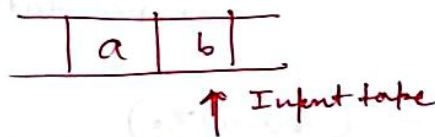
2. POP :-



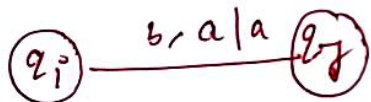
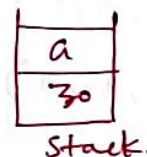
$$\delta(q_i, b, a) = (q_j, \epsilon)$$



3. SKIP :-



$$\delta(q_i, b, a) = (q_j, a)$$



V.V.V.
Imp

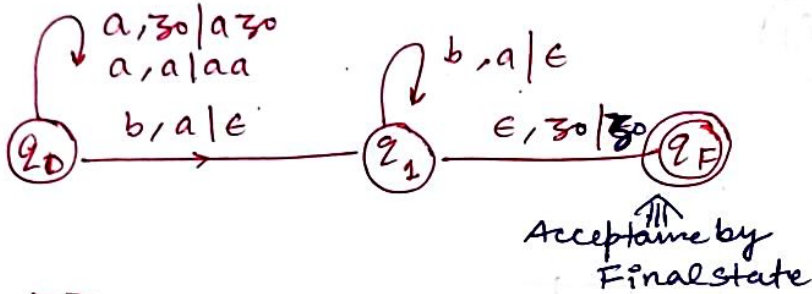
Note :-

1. If PDA Accept string of the language 'L' such that every operation is SKIP operation then PDA is processing without using the external Memory (STACK) Then the PDA is F.A. and language is Regular.

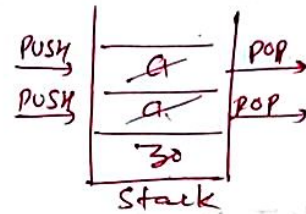
* Construction of PDA :-

1. Construct the PDA For the following language.

$$1. L = \{ a^n b^n \mid n \geq 1 \}$$



$a b$
 $a a b b$
 $a a a b b b$



ID :-

$$\delta(q_0, a, \epsilon) = (q_0, a \epsilon)$$

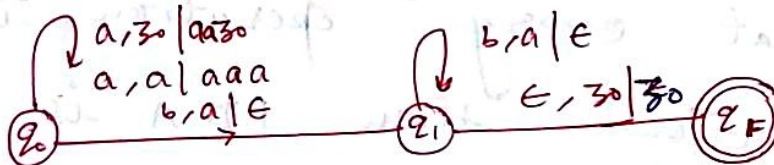
$$\delta(q_0, a, a) = (q_0, aa)$$

$$\delta(q_0, b, a) = (q_1, \epsilon)$$

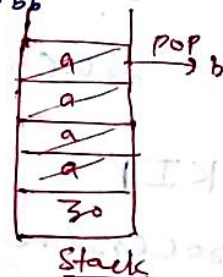
$$\delta(q_1, b, a) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, \epsilon) = (q_F, \epsilon)$$

$$2. L = \{ a^n b^{2n} \mid n \geq 1 \}$$

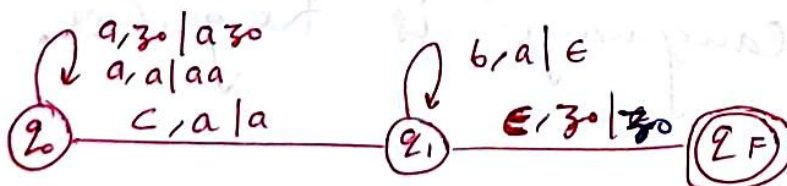


$a b b$
 $a a b b b b$
 $a a a b b b b b$

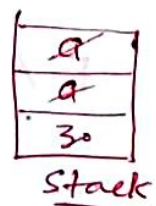


$$3. L = \{ a^n c b^n \mid n \geq 1 \}$$

$\uparrow$ $\uparrow$ $\uparrow$
 PUSH SKIP POP

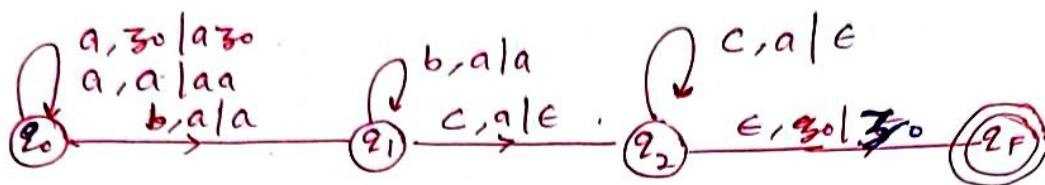


$a c b$
 $a a c b b$
 $a a a c b b b$

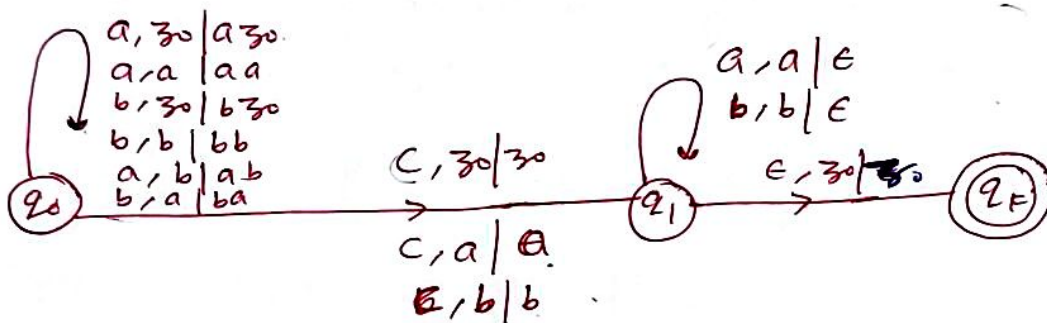


4. $L = \{ a^n b^m c^n \mid n \geq 1, m \geq 1 \}$

$\uparrow$ $\uparrow$ $\uparrow$
 PUSH SKIP POP

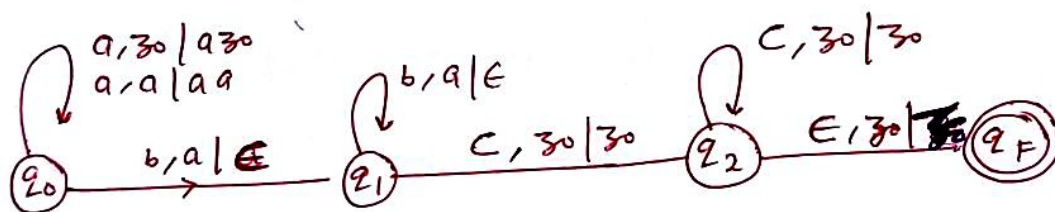


5. $L = \{ w c w^R \mid w \in (a,b)^+ \}$



$\frac{a}{b}$
 $\frac{ab}{ba}$
 $\frac{aabb}{bbaa}$
 $\frac{abab}{baba}$
 $\frac{aaba}{baaa}$

6. $L = \{ a^m b^m c^n \mid m, n \geq 1 \}$



Prefix Property :-

The language 'L' is said to have prefix property if there is no two distinct strings x, y in 'L' such that one is prefix of other.

Note :-

If it is possible to find two distinct strings x, y in 'L' such that one is prefix of other then the language 'L' have no prefix property.

EX :-

1. $L = \{a, ba\} \rightarrow$ has prefix property.

2. $L = \{a, ab\} \rightarrow$ Has no prefix property.

3. $L = \{aba, bab, ab, abb\} \rightarrow$ Has no prefix property.

4. $L = \{a^m b^n \mid m, n \geq 1\} \rightarrow$ Has no prefix property.

5. $L = \{a^m b^n \mid m = n\} \rightarrow$ Has ~~no~~ prefix property.

6. $L = \{w c w^R \mid w \in (a, b)^*\} \rightarrow$ Has prefix property.

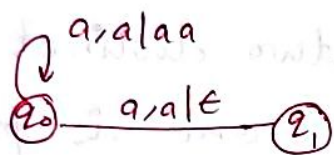
7. $L = \{w w^R \mid w \in (a, b)^*\} \rightarrow$ Has no prefix property.

8. $L = \{w \mid w \in (a, b)^* \mid |w|_a = |w|_b\} \rightarrow$ Has no prefix property.

DCFL :-

The Language 'L' defined over the alphabet ' Σ ' is said to be "DCFL" if $\delta(q, x, y)$ has atmost one outcome for every pair x, y at a state ' q '.

Where $x \rightarrow$ processing symbol from alphabet
 $y \rightarrow$ Topmost symbol of stack.

Ex
1.  $\rightarrow$ No DCFL.

2. $L = \{ w c w^R \mid w \in (a, b)^* \} \rightarrow$ is DCFL

3. $L = \{ w w^R \mid w \in (a, b)^* \} \rightarrow$ CFL

DPDA :-

The PDA is said to be DPDA if $\delta(q, x, y)$ has atmost one move for every pair of processing and top most symbol.

Note :-

1. Every Regular language is Accepted by DPDA. that is every regular language is DCFL, but every DCFL need not be regular.

2. Every language Accepted by DPDA is also Accepted by "NPDA" but converse need not be true.

That is every DCFL is a CFL but every CFL need not be DCFL.



Closure Property of DCFL : —

closed

1. ✓ Compliment
2. ✓ Inverse Homomorphism
3. ✖ Intersection with regular set.
4. ✖ Quotient with regular set.

Not closed.

1. Union.
2. concatenation.
3. clean closure.
4. Intersection.
5. Homomorphism.
6. Substitution.
7. Reverse.

Closure property of CFL

closed

1. Union
2. Concatenation
3. Clean closure
4. Homomorphism
5. Inverse Homomorphism
6. Substitution
7. Reversal
8. Intersection with regular set.
9. Quotient with regular set.

Not closed

1. Intersection
2. Compliment.

Pumping Lemma in CFL :-

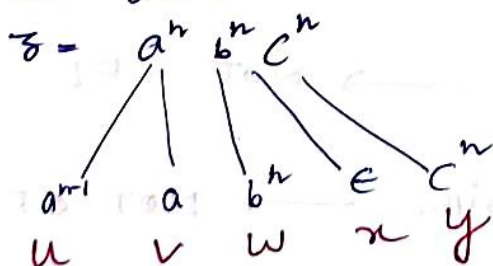
If 'L' is any CFL (Infinite CFL) and $\bar{z} \in L$ such that length of $\bar{z}$, $|\bar{z}| \geq n$ then $uv^iwx^i y \in L$ for all $i \geq 0$ where $\bar{z} = uvwx^i y$, and $|uvx| \leq |\bar{z}|$ and $|vx| \neq 0$.

Note :-

1. Pumping lemma is used to prove that some of the language is not CFL.
2. Every CFL satisfies the pumping lemma property.
3. If 'L' is any language which does not satisfy the pumping lemma property then 'L' is not CFL.
4. A language which satisfies the pumping lemma property need not be CFL.
5. If 'L' is any language and $z \in L$ and $|z| \geq n$ if at least one value for 'i' such that $uv^iwx^iy \notin L$ then 'L' is not CFL.

Ex 1. $L = \{a^n b^n c^n \mid n \geq 1\}$

Let $z \in L$



$$uv^iwx^iy = a^{n-1} a^i b^n (e)^i c^n$$

For $i=2$

$$= a^{n-1} a^2 b^n (e)^2 c^n$$

$$= a^{n+1} b^n c^n$$

$\notin L$

Hence this 'L' is not

CFL

Weak point of Pumping Lemma :-

If 'L' is any language defined over the alphabet $\Sigma = \{a\}$ such that the length of all string in language are in arithmetic progression (A.P) then language 'L' is CFL otherwise not CFL.

Ex :-

1. $L = \{a^{2n} \mid n \geq 1\}$

2, 4, 6, 8 ... A.P $\rightarrow$ CFL

2. $L = \{a^{3n+1} \mid n \geq 1\}$

4, 7, 10, 13 ... A.P. $\rightarrow$ CFL

3. $L = \{a^{n^2} \mid n \geq 1\}$

1, 4, 9 ... NOT A.P $\rightarrow$ NOT CFL

4. $L = \{a^{n^2+5} \mid n \geq 1\} \rightarrow$ NOT CFL

5. $L = \{a^{3^n} \mid n \geq 1\} \rightarrow$ NOT CFL

6. $L = \{a^{n!} \mid n \geq 1\} \rightarrow$ NOT CFL

7. $L = \{a^p \mid p \text{ is prime}\} \rightarrow$ NOT CFL

8. $L = \{a^p \mid p \text{ is not prime}\} \rightarrow$ NOT CFL